

Hackathon Challenge: A standout system feature or an app for a new mobile operating system

Most mobile development today happens inside two dominant ecosystems. HarmonyOS is the first credible challenger in years. It already runs on millions of phones, PCs, watches and other devices, and it is heading beyond Asia. This hackathon is your chance to work with that ecosystem before it arrives here in force.

HarmonyOS is Huawei's own product, and it is proprietary. What makes it reachable for the rest of us is the layer underneath: OpenHarmony, an open-source codebase under vendor-neutral governance of OpenAtom Foundation, and Oniro, its European distribution governed by the Eclipse Foundation. Anyone can read and use the source, ship it on their own hardware, and take it somewhere its original authors never planned.

For Europe, this matters in a practical way. Digital sovereignty means being able to audit, patch and ship the software your products run on, without permission from a vendor on another continent. Oniro gives European companies that control, and the code you write here also runs on HarmonyOS devices, so nothing you build is locked to one roadmap.

So build something bold. Show what a genuinely open mobile platform makes possible when nobody has to ask permission first. Today's agentic AI tools make prototyping a breeze.

Challenge

Your task is to design and build **an innovative system feature or mobile application for an OpenHarmony-based mobile device.**

You are free in how you approach it, but your idea should live in one of the three areas below. Many of the best ideas sit across more than one.

Intelligent Experiences - making products and user experiences intelligent through AI: agents, contextual awareness, personalization, intelligent interaction and on-device AI.

Spatial Experiences - extending digital experiences into physical and three-dimensional space: spatial UI, 3D content, immersive media, sensing, positioning and new forms of interaction with the surrounding environment.

Human-Centric Technology - applying technology to improve human experience, inclusion and quality of life: accessibility, digital wellbeing, inclusive design, education, cultural experiences and responsible technology.

Within these themes we are looking for original ideas, unconventional approaches and ambitious prototypes that explore what an open operating system can enable.

The final solution should demonstrate clear value, a functional implementation and innovation that current platforms are missing.

Technical Requirements

The solution may be implemented using:

- native development using **ArkTS with ArkUI** or **C/C++ platform APIs**;
- a supported cross-platform framework, such as **React Native for OpenHarmony (RNOH)** or another compatible cross-platform framework;
- the relevant OpenHarmony or Oniro system development frameworks and source-level build tools.

The final solution must:

- target **HarmonyOS, OpenHarmony or Oniro**;
- target **API 20 or later** and, where applicable, declare API 20 as the minimum supported API level;
- use an SDK and development environment compatible with the selected platform version;
- run successfully on an OpenHarmony or HarmonyOS emulator or a compatible physical device;
- include reproducible setup, build and launch instructions;
- demonstrate the use or improvement of at least one platform, device or system capability.

The recommended application development setup includes **DevEco Studio, SDK Manager, hviigor, HDC and application signing**, together with an emulator or physical device. This setup is optional: teams may use compatible alternative tools, including the open-source options listed below, provided that the submitted solution can be reproduced and demonstrated.

Cross-platform development is an alternative to native APIs, not a requirement. A cross-platform submission should include a target for OpenHarmony or HarmonyOS, along with the native container, bridge and build configuration required to produce a working platform package; **an Android, iOS, web or desktop build alone is not sufficient.**

An improvement should be delivered as a ready-to-install application or component that adds a new capability to the system or improves an existing one without modifying the system itself.

The submission must clearly explain what the component does, how it integrates with the platform, how it can be installed and how the improvement can be verified.

Use of AI

Coding agents, AI assistants, Model Context Protocol servers, Agent Skills and other AI-enabled development tools and workflows are permitted and strongly encouraged.

AI may run locally, remotely through a service or in a hybrid architecture. Any team whose submission includes an AI feature, or who used AI tools during development, must publish an **AI_WORKFLOW.md** file. For an AI feature, document the model or service, inference flow, data handling, limitations, validation approach and privacy considerations. For development tools, document:

- all AI models, coding agents, MCP servers, Agent Skills and other AI-assisted tools used;
- the main prompts, reusable instructions and relevant configuration;
- the workflow from ideation and architecture through implementation, testing and debugging;
- an explanation of how generated output was reviewed, tested and validated;
- known limitations, unsuccessful approaches and lessons learned.

Prompts and tool usage should be documented as fully as reasonably possible. API keys, credentials, personal data and other confidential information must be removed before publication.

Required Deliverables

Each team must submit:

1. a public source code repository;
2. reproducible setup, build, installation and launch instructions;
3. a working `.hap` package;
4. a brief recorded demonstration;
5. a concise architecture and implementation description;
6. an `AI_WORKFLOW.md` file when AI-assisted development tools were used;
7. additional AI integration documentation when the submission includes AI features.

Evaluation Criteria

Submissions will be assessed based on the criteria below. The notes under each one explain what we look at in practice.

- **Originality - 20%**
 - Is this a new idea, or a fresh take on a known problem?
 - A port of an existing app can be original too, if it solves something non-obvious along the way.
 - Ideas that combine more than one challenge area (for example on-device AI that makes a spatial interface accessible) are a plus, as long as the combination serves a purpose.
- **Demonstrated usefulness of the proposed solution - 20%**
 - Who would use it, and what problem does it solve for them?
 - Which challenge area does it address: Intelligent Experiences, Spatial Experiences or Human-Centric Technology? The connection should be visible in what the solution does, not only in how it is described.
 - We value a working, narrow solution to a real problem over a broad concept that only exists on slides.

- **Technical execution - 20%**

- Does it actually work as described? Claims should be backed by the code, the demo, logs or test results.
- Is the architecture sensible: are the components, data flows and integrations justified, not added for show?
- Is the code readable and modular, with reasonable error handling?
- How does it behave when things go wrong: API errors, timeouts, missing data, unexpected input or, if you use AI models, incorrect model output?
- Are there tests covering the key scenarios? Full coverage is not expected, but some evidence that you checked your own work is.
- Basic hygiene matters: no secrets in the repo, input validation, no unnecessary permissions or risky dependencies.

- **Use or enhancement of platform capabilities - 20%**

- Does the solution make real use of the platform, for example its system services, APIs or distributed features? An app that would run unchanged on any other OS, without touching anything platform-specific, scores lower here. Cross-platform frameworks are fine, as long as the solution also uses what the platform itself provides.
- Depending on the area, this may mean on-device AI and agent frameworks, sensors, positioning and 3D rendering, or accessibility and system services.

- **Quality of the demonstration - 10%**

- Can we see the solution actually running, not just mockups? The emulator is the expected default.
- Is it clear what we are looking at and what part was built during the hackathon?
- Show the core interaction as realistically as you can. If part of it can't run on the emulator (sensors, positioning, physical context), explain clearly how it would work. Mentors have devices on site if you want to try it on real hardware.

- **Reproducibility and transparency of the development workflow - 10%**

- Can someone else build and run it from the README and the repository alone?
- Are dependencies and environment setup documented (versions, SDKs, configuration)?
- Does the commit history show how the work progressed? If you used AI tools, describe how. That's fine, we just want to see it.

Repositories may undergo an automated technical pre-review; the final assessment is made by the jury.